

EE 446: TinyML Lab 7

On-Device Data Collection and Tiny Ensemble Learning

Luke Valerio · Student ID 1901711 · Summer 2026

This report covers both parts of Lab 7. Part I is on-device IMU data collection and motion classification between an idle state and a lifting motion, built in Edge Impulse and deployed to the Arduino Nano 33 BLE Sense. Part II is a tiny ensemble-learning pipeline built in `TinyML_Lab7_Part_II.ipynb`. It trains three autoencoder-and-classifier branches over three different representations of the mHealth IMU data, combines them with a stacked meta-classifier, then compresses the result with pruning and quantization-aware training and exports int8 TFLite models for deployment.

Section 1. Part I: Edge Impulse Motion Classification

Public Edge Impulse project link

<https://studio.edgeimpulse.com/public/1084899/live>

Data collection

Device	Arduino Nano 33 BLE Sense
Sensor	Inertial (IMU)
Classes	<code>idle</code> , <code>lift</code>
Samples per class	20
Sample length	10 s each
Sampling rate	100 Hz

Each 10-second recording carries a single label, and the impulse slides a fixed-length window across it, so one recording produces many labeled training windows rather than one. The lifting motion was therefore repeated continuously through each recording. A single isolated lift would leave most windows in that recording stationary but still labeled `lift`, and those windows would be indistinguishable from the `idle` class while carrying the opposite label.

Impulse design

The impulse is a time-series input block, followed by a Spectral Analysis processing block, followed by a neural network classifier with two outputs. The spectral block converts each window into RMS, peak-frequency, and spectral-power features, so the classifier sees a compact frequency-domain description of the motion rather than the raw waveform. This suits a repeated, quasi-periodic motion such as lifting, where the distinguishing information is the frequency content of the window rather than the exact shape or timing of any single gesture.

Model performance

Metric	Value
Validation accuracy	96.7%
Test accuracy	91.0%
Estimated on-device latency (Cortex-M4F at 64 MHz)	32 ms
Estimated peak RAM	3.1 KB

The gap between validation accuracy at 96.7% and test accuracy at 91.0% is worth noting rather than glossing over. Consecutive windows drawn from the same recording overlap heavily, so a window in the validation split is often a near neighbor of a window the model trained on. That inflates the validation figure. The test split is held out at the recording level, so 91.0% is the more honest estimate of how the model generalizes to a new recording.

Deployment evidence

```
lift: 0.910198
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 39 ms, inference 485 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 39 ms, inference 486 us, anomaly 0 ms, postprocessing 76 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 39 ms, inference 516 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.199219
  lift: 0.800781
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 33 ms, inference 514 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 33 ms, inference 542 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.968750
  lift: 0.031250
Starting inferencing in 2 seconds...
Sampling...
```

Figure 1. Live inference on the Arduino Nano 33 BLE Sense via `edge-impulse-run-impulse`. The board correctly identifies the lifting motion (lift 0.996094 across several consecutive windows) and the resting state (idle 0.968750), with one transitional window at lift 0.800781 and idle 0.199219 captured as the motion stops. Per-window timing: DSP 33 to 39 ms, classifier inference 485 to 542 μ s, post-processing 48 to 76 μ s.

READING THE DEPLOYED OUTPUT

Two things in this trace are worth drawing out.

First, the DSP block dominates the compute, not the neural network. Feature extraction takes 33 to 39 ms while the classifier itself runs in roughly 0.5 ms, so spectral analysis is on the order of 70 to 80 times more expensive than inference. The measured DSP time also lines up with the 32 ms latency that Edge Impulse estimated for this board, which confirms the deployed impulse is the one that was trained. The practical lesson for a TinyML system is that shrinking the model architecture would buy almost nothing here, because nearly all of the per-window cost sits in the front end.

Second, the confidence values expose the quantization. Every printed score is an exact multiple of $1/256$: 0.996094 is $255/256$, 0.968750 is $248/256$, 0.800781 is $205/256$, and 0.199219 is $51/256$. That is the signature of an int8 output tensor with a scale of $1/256$, which is the same quantization idea applied by hand to the Part II models. It also explains why two class scores sometimes sum to 0.996094 rather than exactly 1.0, since the rounding residue has nowhere to go.

The transitional window at lift 0.800781 and idle 0.199219 is the model behaving sensibly rather than failing, because that window straddles the moment the lifting stopped and genuinely contains both states. It is also worth noting that `run-impulse` in this mode samples a fresh window each cycle rather than sliding an overlapping buffer, so consecutive predictions in this trace are independent windows rather than the overlapping stride used to generate training data.

Section 2. Part II: Tiny Ensemble Learning Results

Environment

```
Python executable: /Users/lukevalerio/ai/projects/tinyml-arduino/bin/python
Python version:    3.11.15
TensorFlow:        2.14.1
TF-MOT:            0.8.0
```

Data preparation

The source file is `mHealth_subject6.log`, containing 98,304 rows. Rows carrying label 0, the null or unlabeled class, are dropped, which leaves 32,205 labeled samples across 12 activity classes. Six IMU channels are used, being three accelerometer axes in m/s^2 and three gyroscope axes in rad/s , and the labels are converted from 1-based to 0-based indexing.

Three parallel copies of the same signal are created to give the ensemble its branch diversity: raw, standard-scaled to zero mean and unit variance, and min-max-scaled to the range $[0, 1]$. Each copy is windowed into 100-timestep windows with stride 1 and flattened to 600 values. The 70/30 train and test split is applied within each class, so the temporal ordering inside each activity segment is preserved and no window straddles the split boundary.

Representation	Train windows	Test windows	Input dimension
Raw	21,350	8,479	600
Standard-scaled	21,350	8,479	600
Min-max-scaled	21,350	8,479	600

Input-space visualization

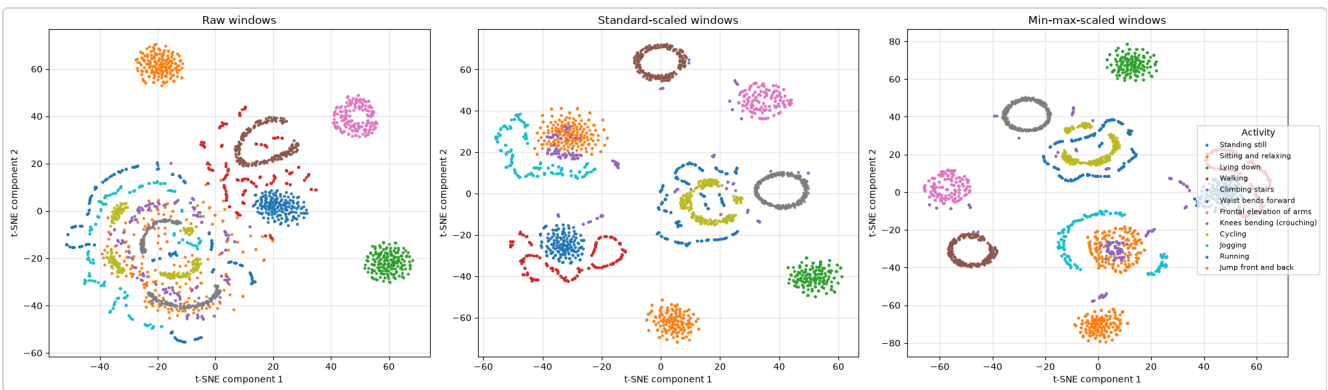


Figure 2. t-SNE of the 600-dimensional test windows under the three input representations, using a stratified subset of roughly 2,000 windows. All three already show well-separated activity clusters, which is why the branch classifiers reach high accuracy. The three views are not identical, and that difference is the diversity the ensemble exploits.

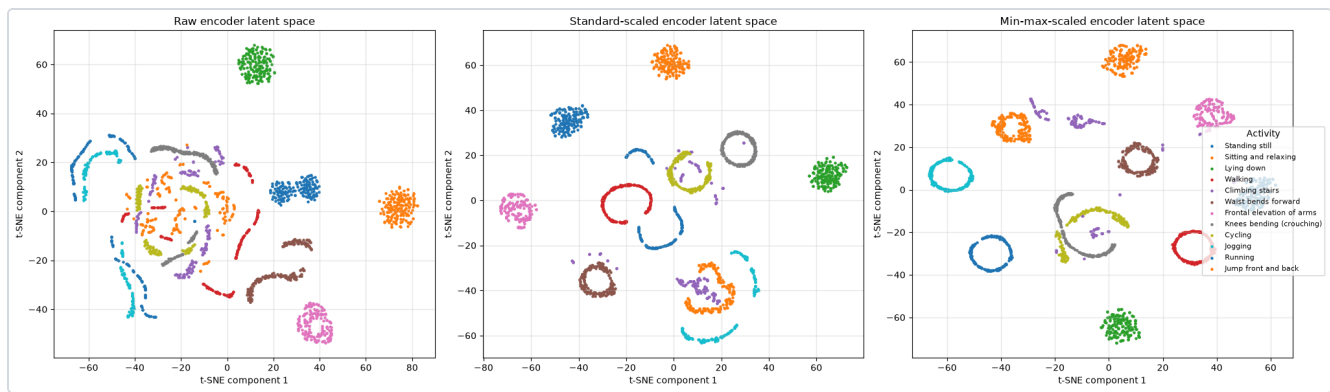


Figure 3. t-SNE of the 32-dimensional encoder outputs for the same windows. The encoders compress 600 values down to 32, a reduction of 18.75 times, while keeping the activity clusters distinct. This confirms the latent vector is a usable feature representation for the classifiers.

Branch classifier performance in float32

Branch	Test accuracy	Macro-average F1
Raw	0.99	0.99
Standard-scaled	0.99	0.99
Min-max-scaled	0.97	0.97
Stacked ensemble	1.00	1.00

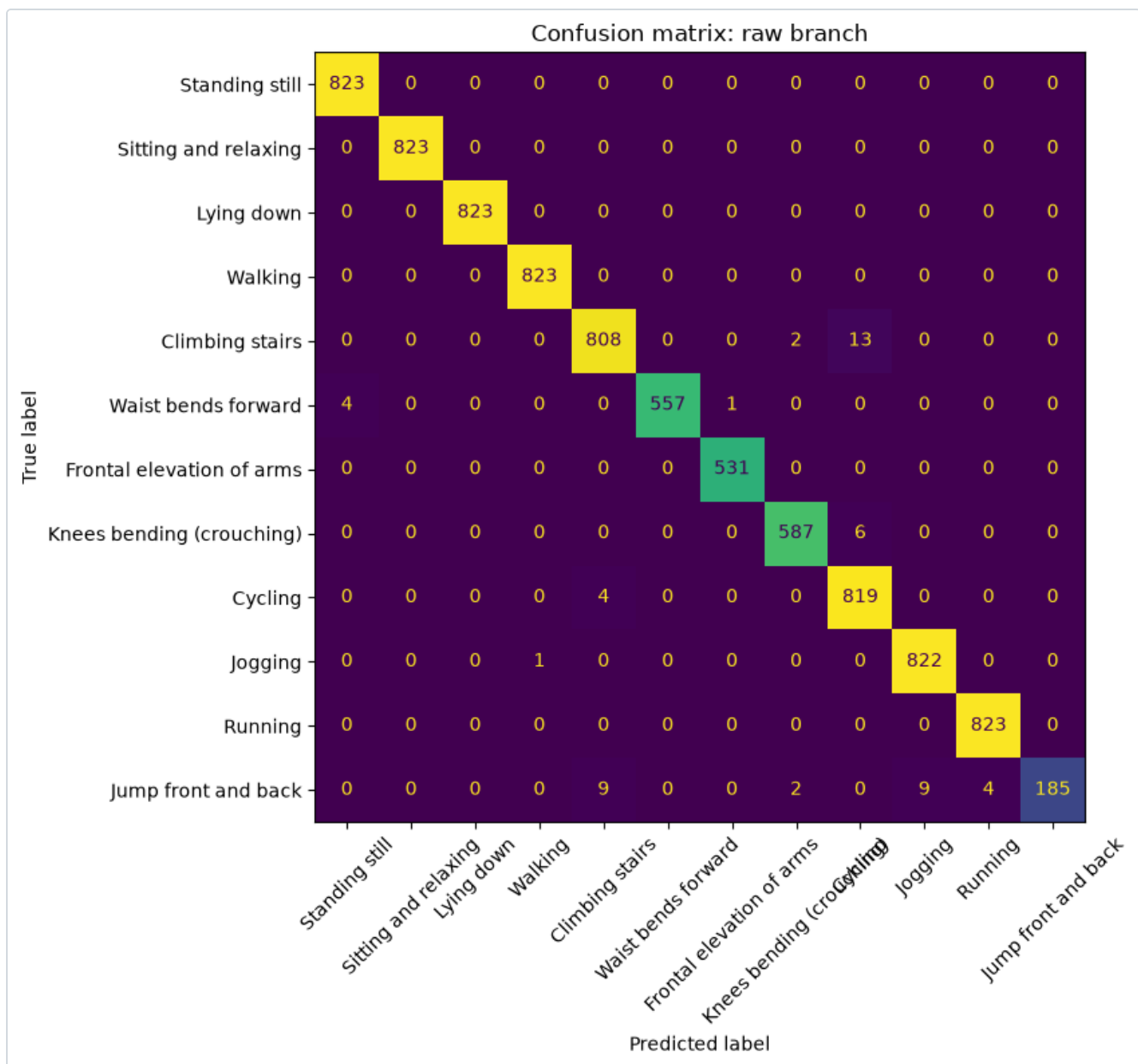


Figure 4. Confusion matrix for the raw branch, accuracy 0.99.

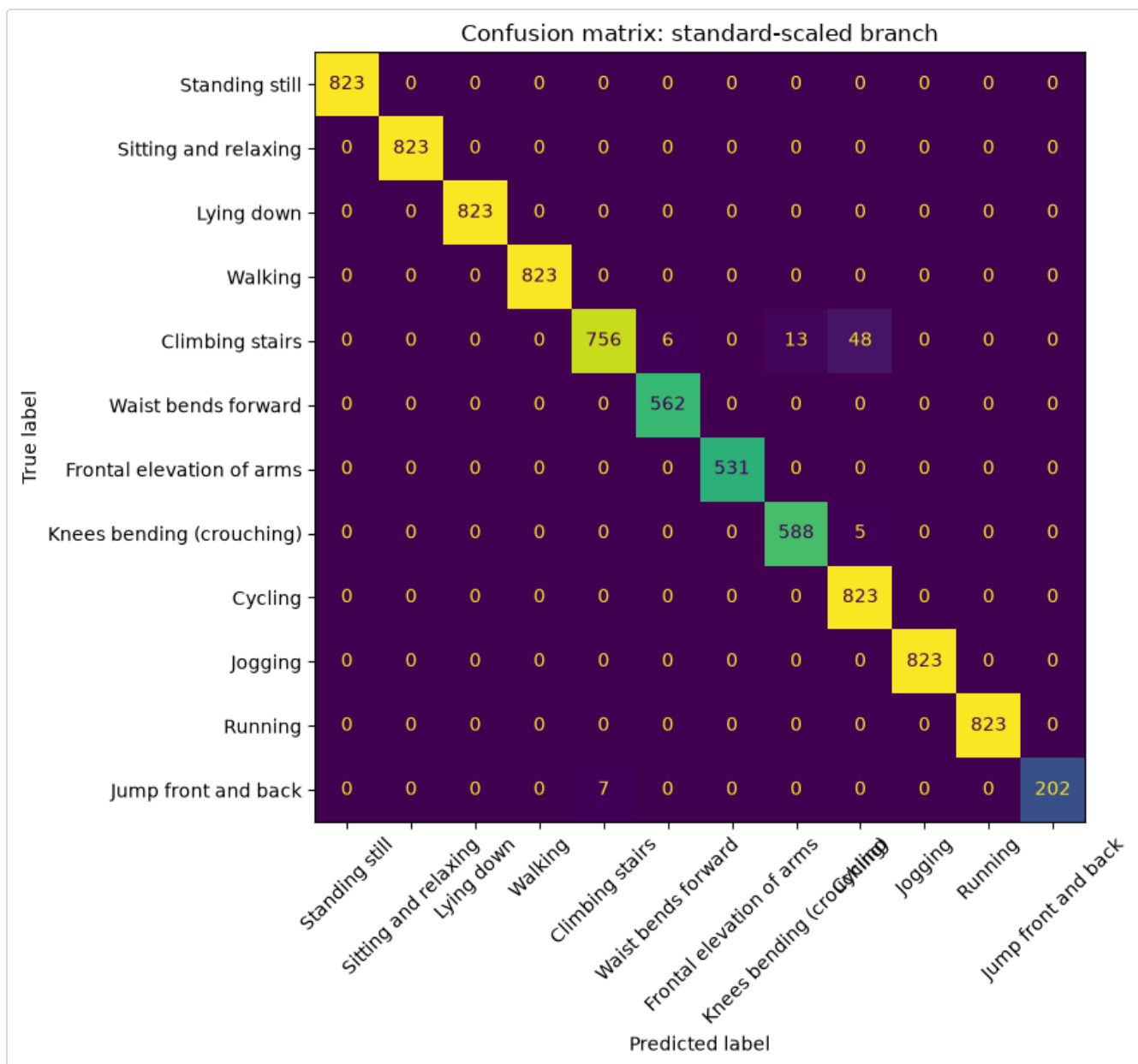


Figure 5. Confusion matrix for the standard-scaled branch, accuracy 0.99.

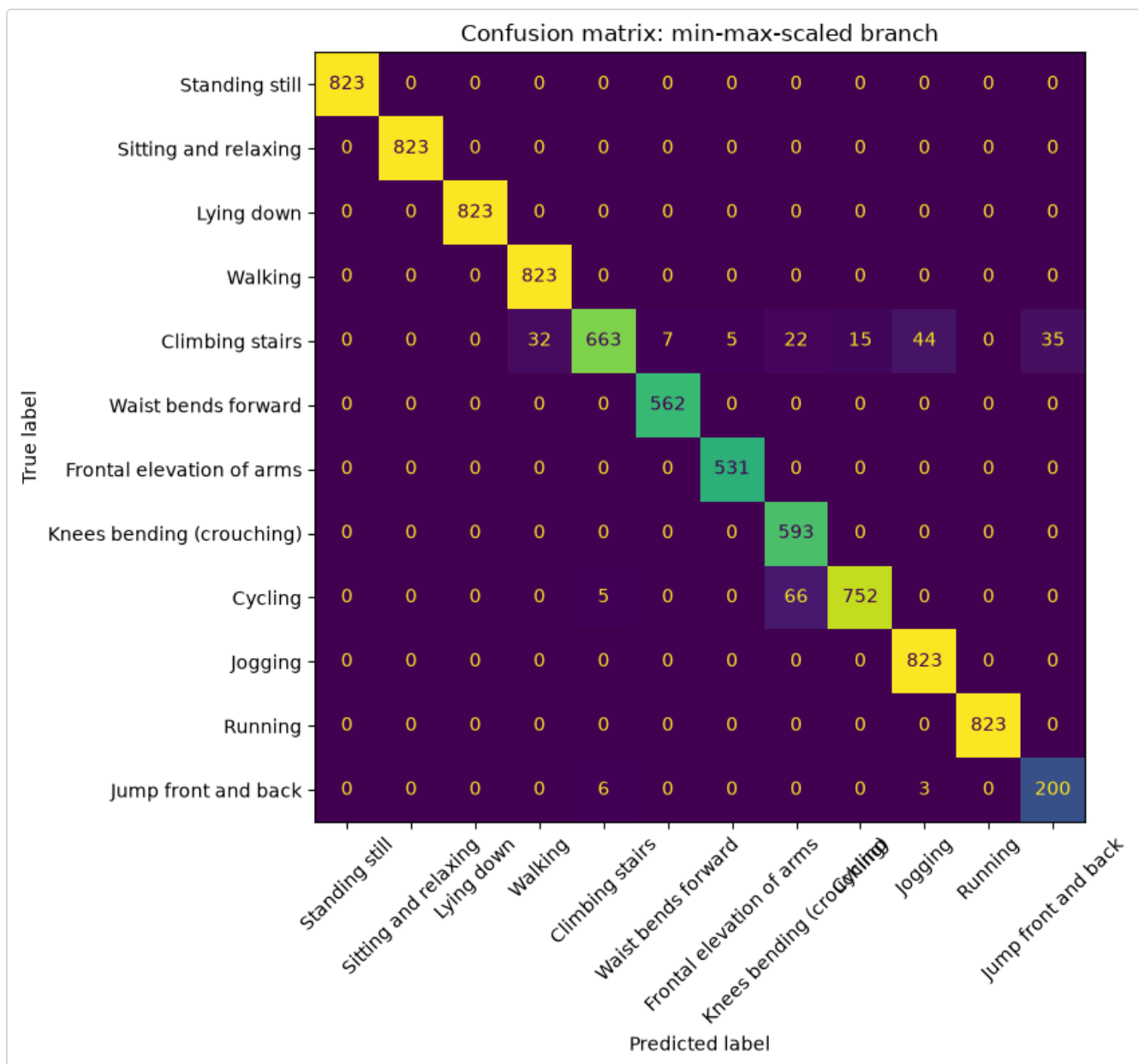


Figure 6. Confusion matrix for the min-max-scaled branch, accuracy 0.97. This is the weakest branch. Min-max scaling compresses the dynamic range of the high-motion activities, so classes such as jogging and running become harder to separate.

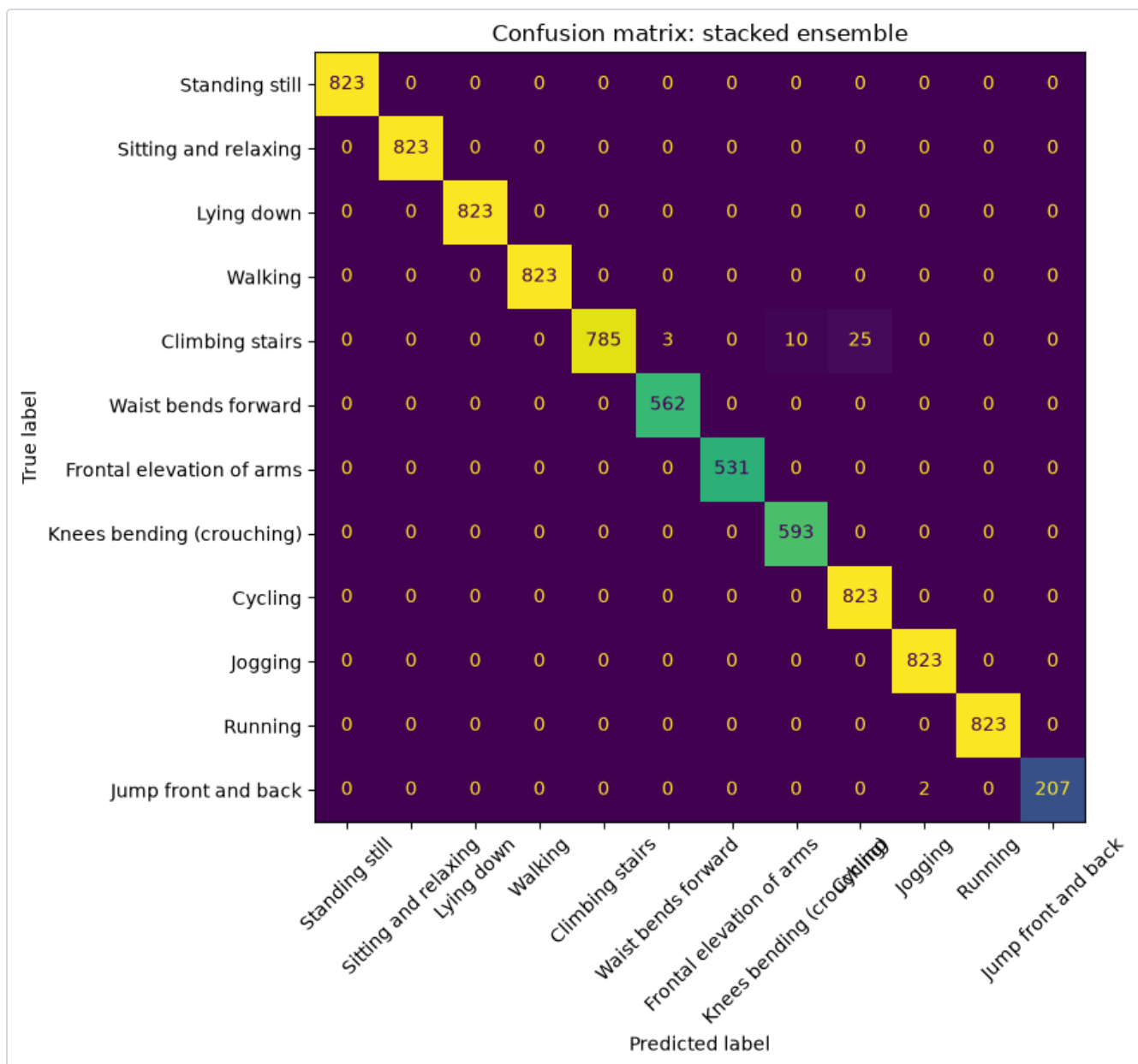


Figure 7. Confusion matrix for the stacked meta-classifier, accuracy 1.00. Stacking recovers the errors that any single branch makes, because a window misclassified under one representation is usually still correct under the other two.

Baseline float32 TFLite sizes

Model	Size (KB)
encoder_raw, encoder_std, encoder_minmax	159.88 each
clf_raw, clf_std, clf_minmax	5.41 each
stacked_meta_clf	6.50
Total across all seven float32 models	502.37

Compression results with pruning, QAT, and int8

Model	Float32 accuracy	int8 accuracy	Sparsity	int8 size (KB)
encoder_clf_raw	0.99	0.9220	79.44%	44.55
encoder_clf_std	0.99	0.9901	79.44%	44.55
encoder_clf_minmax	0.97	0.9322	79.44%	44.61
stacked_meta_clf	1.00	0.8948	79.43%	3.63

Layer-wise sparsity is essentially uniform across every Dense layer, which is what a global schedule applied per layer should produce:

```
quant_encoder_dense_64    : 79.44% sparse (30505/38400)
quant_latent_32           : 79.44% sparse ( 1627/ 2048)
quant_classifier_dense_20 : 79.38% sparse (  508/  640)
quant_activity_softmax    : 79.58% sparse (  191/  240)
Overall                   : 79.44% sparse (32831/41328)
```

Each flattened branch drops from 165.29 KB in float32, being 159.88 KB of encoder plus 5.41 KB of classifier, to 44.55 KB as a pruned int8 model. That is a reduction of 3.7 times. All four deployed models together occupy roughly 137 KB.

One caveat should be stated plainly. The size reduction comes almost entirely from int8 quantization and not from pruning. TFLite stores dense weight tensors in full, so a weight that has been zeroed still costs one byte on disk. The 79.44% sparsity produces no file-size saving here, and it would only pay off with a sparse kernel or a compressed storage format. Its practical benefit in this lab is regularization and preparation for hardware that can exploit sparsity.

Scaling parameters exported for the Arduino sketch

Feature	Mean	Std	Min	Max
Accel_X	3.268805	4.253204	-22.091000	20.003000
Accel_Y	-9.153526	5.397565	-19.570999	20.909000
Accel_Z	0.700608	6.433631	-19.363001	24.599001
Gyro_X	-0.020571	0.325429	-0.977740	1.463800
Gyro_Y	-0.520382	0.571098	-1.611600	1.557200
Gyro_Z	-0.299817	0.542104	-1.206300	1.326100

These match the constants already hardcoded in `Tiny_Ensemble_Learning.ino` at lines 70 to 73, so no edit to the sketch was required. The mean of -9.15 m/s² on Accel_Y is the signature of gravity lying along the sensor's Y axis, which is a direct consequence of how the mHealth sensor was mounted. That detail returns in Question 4.

Build verification

The four generated C arrays were copied into the sketch folder, and the sketch compiles for the target board:

```
$ arduino-cli compile --fqbn arduino:mbed_nano:nano33ble Tiny_Ensemble_Learning
Sketch uses 414688 bytes (42%) of program storage space. Maximum is 983040 bytes.
Global variables use 194992 bytes (74%) of dynamic memory, leaving 67152 bytes
for local variables. Maximum is 262144 bytes.
```

Section 3. Discussion Questions

Question 1: Model architecture and ensemble flow

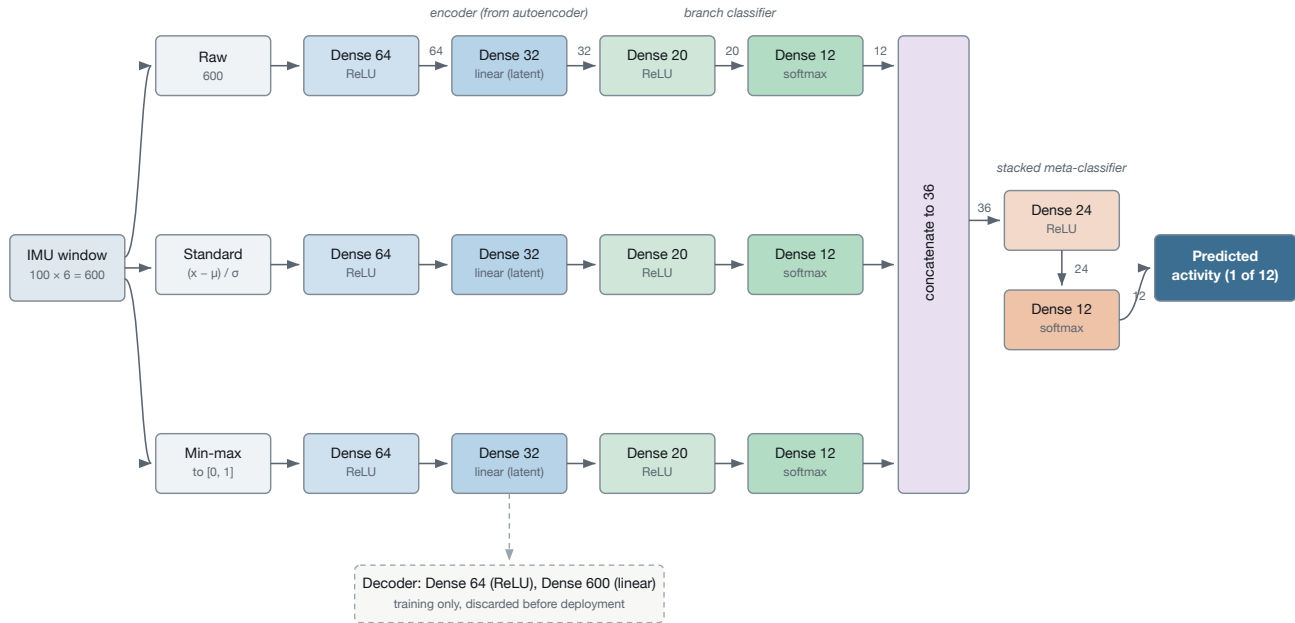


Figure 8. Complete ensemble pipeline. Each of the three branches is an independent encoder and classifier chain. The three 12-class softmax vectors concatenate into a 36-dimensional vector that feeds the stacked meta-classifier. The decoder half of each autoencoder is used only to train the encoder and is discarded before deployment.

INPUT

Each sample is a window of 100 time steps by 6 IMU features, being accelerometer X, Y, Z and gyroscope X, Y, Z. The window is flattened row-major into a single vector of 600 input neurons. At the 50 Hz sampling rate of the mHealth data this represents a 2-second window.

THE THREE BRANCHES

The same 600-value window is presented in three representations: raw, standard-scaled, and min-max-scaled. Every branch has an identical topology, and only the input scaling and the learned weights differ. This is what creates ensemble diversity without needing three different architectures or three different label sets.

ENCODER IN EACH BRANCH

$600 \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(32, \text{linear})$. Each encoder is the first half of an autoencoder trained for 50 epochs with MSE loss and Adam at a learning rate of 0.001 to reconstruct its own input. The matching decoder is $32 \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(600, \text{linear})$. Once training finishes the

decoder is discarded and the encoder is reused as a fixed feature extractor producing a 32-dimensional latent vector.

CLASSIFIER IN EACH BRANCH

`32 → Dense(20, ReLU) → Dense(12, softmax)` , trained for 50 epochs with categorical cross-entropy and Adam at a learning rate of 0.001. The 12 outputs correspond to the 12 mHealth activity classes.

ENSEMBLE CONNECTION

Three models feed the ensemble, one per representation. Each emits a 12-dimensional softmax vector, and the three are concatenated as 12 plus 12 plus 12 into the 36-dimensional meta-classifier input.

STACKED META-CLASSIFIER

`36 → Dense(24, ReLU) → Dense(12, softmax)` . It learns how much to trust each branch for each class rather than simply averaging their votes, which is why it can outperform all three of them individually. Counting the meta-classifier, the full ensemble is four trained networks at inference time, plus three decoders that exist only during training.

For deployment, each encoder and its classifier are merged into a single flat model, `600 → Dense(64, ReLU) → Dense(32, linear) → Dense(20, ReLU) → Dense(12, softmax)` , so the board runs exactly four TFLite interpreters: the three branches, then the meta-classifier.

Question 2: Pruning and QAT methodology

HOW PRUNING IS APPLIED AND THE SPARSITY SCHEDULE

`prune_low_magnitude()` wraps every Dense layer in a `PruneLowMagnitude` wrapper that adds a binary mask and a threshold variable alongside the kernel. During training the wrapper ranks each layer's weights by absolute value and zeroes the smallest fraction, following a `PolynomialDecay` schedule that runs from an initial sparsity of 10% to a final sparsity of 80%, with `begin_step` at 0, `end_step` at 500, and the default cubic ramp. Training runs for 5 epochs at batch size 256.

This explains why the measured result is 79.44% rather than a clean 80%. With 21,350 training windows at batch size 256 there are 84 steps per epoch, so 5 epochs is 420 steps and the schedule never reaches step 500. TF-MOT also recomputes masks only every 100 steps by default, so the last mask update happens at step 400:

```
sparsity(400) = 0.80 + (0.10 - 0.80) × (1 - 400/500)3
               = 0.80 - 0.70 × 0.008
               = 0.7944, which is exactly the measured value
```

The `UpdatePruningStep()` callback is what advances the internal step counter. Without it the schedule would never progress at all.

WHY THE PRUNING WRAPPERS ARE STRIPPED BEFORE QAT

`strip_pruning()` removes the wrapper objects along with their mask and threshold bookkeeping variables, returning a plain Keras model whose kernels simply contain zeros where weights were pruned. This is necessary because `quantize_model()` does not know how to quantize a `PruneLowMagnitude` wrapper. It expects standard Keras layers, and would either fail or carry dead pruning variables into the TFLite graph. Stripping bakes the sparsity into the weight values themselves and leaves a clean graph for the quantizer.

HOW QAT IS APPLIED AFTER PRUNING

`quantize_model()` wraps each remaining layer in a `QuantizeWrapper` that inserts fake-quantize operations on weights and activations. During the 5 QAT epochs the forward pass rounds values to the int8 grid while gradients still flow in float via a straight-through estimator, so the network learns weights that remain accurate after rounding. The wrappers also record running minimum and maximum activation ranges, which become the scale and zero-point in the exported model.

WHY THE EXTRA MASK-ENFORCEMENT STEP IS NEEDED

This is the key subtlety in the pipeline. Once the pruning wrappers are stripped, nothing holds the pruned weights at zero. QAT is ordinary gradient training with Adam, and a weight sitting at exactly 0.0 still receives a nonzero gradient, so it immediately drifts away from zero. Five epochs of unconstrained QAT would silently refill most of the 79.44% sparsity, and the model described as pruned would no longer be sparse at all.

`extract_dense_weight_masks()` therefore snapshots a binary mask of every nonzero weight right after stripping, and `MaskEnforcerCallback` re-multiplies the QAT model's weights by those masks at the end of every training batch and every epoch, then once more after `fit()` returns. This turns QAT into a projected-gradient procedure, where the optimizer proposes an update and the callback projects it back onto the sparse subspace. The verified 79.44% sparsity in the final model is the evidence that it worked.

INT8 TFLITE CONVERSION AND WHY REPRESENTATIVE DATA IS NEEDED

Conversion sets `optimizations = [Optimize.DEFAULT]`, restricts `supported_ops` to `TFLITE_BUILTINS_INT8`, and forces both `inference_input_type` and `inference_output_type` to `tf.int8`, producing a model with no float operators anywhere.

The converter needs a representative dataset, here 200 evenly spaced training windows, because weights are known and can be measured directly but activations cannot. Activation ranges depend entirely on what data flows through the network. The converter runs the representative samples through the model, observes the real minimum and maximum at every intermediate tensor, and derives the scale and zero-point that map each float range onto the interval from -128 to 127. Poor calibration data means clipped or low-resolution activations and a sharp accuracy drop, which is why the representative set must resemble real deployment input.

WHY THIS MATTERS ON THE ARDUINO NANO 33 BLE SENSE

Each branch shrinks from 165.29 KB in float32 to 44.55 KB in int8, a reduction of 3.7 times, and the whole four-model ensemble fits in roughly 137 KB. The compiled sketch uses 42% of flash and 74% of RAM, so it fits, but without a great deal of headroom, which is exactly the constraint this pipeline exists to satisfy. Beyond raw size, int8 tensors quarter the arena footprint, and the arena is the binding constraint here. Sparsity contributes regularization and would give further gains on hardware or kernels that skip zero weights, though as noted above it does not shrink the `.tflite` file on its own.

The accuracy cost is visible and worth reporting honestly. The standard-scaled branch survives almost untouched, moving from 0.99 to 0.9901, but the raw branch falls to 0.9220 and the min-max branch to 0.9322. The meta-classifier suffers most, dropping from 1.00 to 0.8948, because it is by far the smallest model at 1,152 weights and has the least redundancy to give away, and because it was trained on nearly perfect branch outputs and therefore meets a regime it never saw once those branch outputs degrade.

Question 3: Arduino deployment and real-world behavior

Pending: hardware observation required

The Part I evidence in Section 1 comes from the Edge Impulse idle and lift model. This question concerns the separate Part II ensemble sketch, which classifies the 12 mHealth activities. Upload

`Tiny_Ensemble_Learning.ino`, which already compiles, let the board rest flat, and capture the serial monitor at 115200 baud. Replace the placeholder below, then confirm or correct the analysis that follows, which is written from the code and the training data and has not yet been checked against live output.

Screenshot: serial monitor output with the board at rest

Show several consecutive windows including the per-branch score lines and the final prediction

IS THE OUTPUT ACCEPTABLE FOR A RESTING BOARD?

For a board sitting still, the defensible answers are standing still, sitting and relaxing, or lying down. All three are near-stationary classes, and the IMU cannot really distinguish them from a desk. A stable prediction among those three is acceptable. A confident and unchanging prediction of a high-motion class such as jogging, running, or cycling is not acceptable, and would indicate a distribution mismatch rather than a working classifier.

ISSUES TO EXPECT AND CHECK

- Orientation and mounting mismatch. The training statistics show a mean of -9.15 m/s^2 on `Accel_Y`, meaning gravity lay along the sensor's Y axis in the mHealth recordings. A Nano lying flat on a desk puts gravity on Z instead, so every window is offset from the training distribution before the model starts.
- Prediction instability. The sketch classifies each 100-sample window independently with no smoothing, so borderline windows can flip label every 2 seconds.
- Scaling and unit handling. The sketch does this part correctly, converting g to m/s^2 and deg/s to rad/s and using the same statistics printed by the notebook. It is still worth confirming that the printed branch scores look like real distributions rather than saturated one-hot vectors.
- Sensor noise and sampling. The sketch samples on a 20 ms timer to match the 50 Hz training rate, but it reuses the most recent available reading rather than waiting for a fresh one, so a slower IMU output rate produces duplicated samples and an artificially smooth window.

TWO PRACTICAL CHANGES THAT WOULD IMPROVE DEPLOYMENT RELIABILITY

1. Train on data from the deployment sensor. Record with `Raw_IMU_Recorder.ino` in the actual mounting position, then either fine-tune the branches on that data or at minimum recompute the scaler statistics from it. This addresses the root cause rather than the symptom.
2. Add temporal smoothing and a confidence gate. Take a majority vote over the last three to five windows and suppress any prediction whose meta-classifier confidence falls below a threshold,

reporting an uncertain state instead. This removes single-window flicker and stops the board from asserting a confident label on out-of-distribution input.

Question 4: Deployment behavior on the Arduino

Pending: observation to confirm

State whether the model did in fact settle on one class while the board was moved, and which class. The explanation below covers the likely causes, and the ones that match the observed behavior should be kept.

A model that predicts the same class regardless of how the board moves is the expected failure mode here, and it is a data mismatch problem rather than a bug in the model or the conversion. The pipeline reached 100% test accuracy in the notebook, so the network itself is sound. What changed is the input distribution.

DIFFERENT SENSOR AND DIFFERENT BODY LOCATION

The models were trained on one subject's mHealth recording, captured by body-worn Shimmer sensors on the chest and ankles. A Nano 33 BLE Sense held in the hand or lying on a desk produces signals from an entirely different sensor at an entirely different body location. Walking measured at the ankle looks nothing like walking measured by a board resting in a palm.

AXIS ORIENTATION AND SIGN CONVENTIONS

The mHealth sensor axes do not line up with the Nano's silkscreen axes. With gravity on the training set's Y axis and on the board's Z axis, the input vector is effectively a permuted version of what the encoder expects, and a dense network has no invariance to that permutation.

UNIT CONVERSION

The training data is in m/s^2 and rad/s while the BMI270 library returns g and deg/s . The inference sketch handles this correctly through `kGToMs2` and `kDegToRad`. It is worth flagging that

`Raw_IMU_Recorder.ino` does not apply these conversions, so a CSV recorded with it is off by a factor of 9.81 on the accelerometer and 57.3 on the gyroscope. Feeding that file directly into the optional Section 18 validation would produce meaningless predictions unless the units are converted first.

FEATURE SCALING CALIBRATED TO A SINGLE SUBJECT

The mean, standard deviation, minimum, and maximum constants all come from subject 6 alone. Any live reading outside that recorded range, which is easy to produce because min-max scaling was fit on one subject's motion envelope, is pushed outside the interval $[0, 1]$ for the min-max branch and into a region the encoder never saw during training.

BRITTLINESS OF THE STACKED META-CLASSIFIER

This cause is specific to the ensemble design and is probably the dominant effect. The meta-classifier was trained on branch outputs that were already essentially perfect, so its 36-dimensional input was always three near-one-hot vectors that agreed with each other. It never saw diffuse or contradictory branch outputs. On out-of-distribution board data the branches disagree and produce flat softmax vectors, and

the meta-classifier has no learned response for that region, so it collapses onto whichever class its bias terms favor and stays there. The int8 quantization compounds the problem, since the meta-classifier already lost the most accuracy of the four models, falling from 1.00 to 0.8948.

MOTION-PATTERN MISMATCH

The 12 mHealth classes are sustained, whole-body activities recorded over minutes. Waving a board around by hand for a few seconds does not resemble any of them, so no correct answer exists for that input. The model is being asked a question outside its domain, and it answers with whatever its decision surface extrapolates to.

Section 4. Deliverables and Status

Generated artifacts

File	Description
<code>TinyML_Lab7_Part_II.ipynb</code>	Notebook with all 18 code cells executed and outputs populated
<code>encoder_clf_raw_pruned_qat_int8.tflite</code>	Raw branch, pruned and QAT int8, 44.55 KB
<code>encoder_clf_std_pruned_qat_int8.tflite</code>	Standard-scaled branch, 44.55 KB
<code>encoder_clf_minmax_pruned_qat_int8.tflite</code>	Min-max-scaled branch, 44.61 KB
<code>stacked_meta_clf_pruned_qat_int8.tflite</code>	Stacked meta-classifier, 3.63 KB
Matching <code>.cc</code> arrays	C arrays generated with <code>xxd -i</code> , placed in the sketch folder
<code>Tiny_Ensemble_Learning.ino</code>	Deployment sketch, verified to compile for the Nano 33 BLE

Status

Item	Status
Part I Edge Impulse project, trained and public	Complete
Part I serial evidence for both idle and lift	Complete
Part II notebook executed with outputs populated	Complete
Part II int8 models and C arrays generated	Complete
Part II sketch compiles for the target board	Complete
Question 1 architecture diagram and description	Complete
Question 2 pruning and QAT explanation	Complete
Question 3 serial screenshot from the ensemble sketch	Pending
Question 4 observed behavior on the board	Pending
Optional recorded-IMU validation in Section 18	Not attempted

Report generated from `TinyML_Lab7_Part_II.ipynb`, executed on 2026-08-10 in the local `tinym1-arduino` environment.